

Operating Systems - Complete GATE Preparation Guide

Table of Contents

1. [Introduction to Operating Systems](#1-introduction-to-operating-systems)
2. [Process Management](#2-process-management)
3. [CPU Scheduling](#3-cpu-scheduling)
4. [Process Synchronization](#4-process-synchronization)
5. [Deadlock](#5-deadlock)
6. [Memory Management](#6-memory-management)
7. [Virtual Memory](#7-virtual-memory)
8. [File Systems](#8-file-systems)
9. [I/O Systems](#9-io-systems)
10. [Important Formulas and Quick Reference](#10-important-formulas-and-quick-reference)

1. Introduction to Operating Systems

What is an Operating System?

An operating system is a program that acts as an interface between computer hardware and user applications. It manages hardware resources and provides services to applications.

Key Functions:

- **Process Management**: Creating, scheduling, and terminating processes
- **Memory Management**: Allocating and deallocating memory
- **File Management**: Managing files and directories
- **Device Management**: Managing I/O devices
- **Security**: Protecting system resources

Types of OS:

- **Batch OS**: Jobs processed in batches (no interaction)
- **Interactive OS**: Direct user interaction
- **Time-sharing OS**: Multiple users share CPU time
- **Real-time OS**: Responses within strict time constraints
- **Distributed OS**: Resources spread across multiple machines
- **Embedded OS**: Designed for specific hardware

System Calls:

Interface between user programs and OS kernel. Types:

- Process control (fork, exec, wait, exit)
- File operations (open, read, write, close)
- Device management
- Information maintenance
- Communication

2. Process Management

Process vs Program:

- **Program**: Static code stored on disk
- **Process**: Program in execution with associated resources

Process States:

1. **New**: Process being created
2. **Ready**: Process waiting for CPU
3. **Running**: Instructions being executed
4. **Waiting/Blocked**: Process waiting for I/O or event
5. **Terminated**: Process finished execution

Process Control Block (PCB):

Contains process information:

- Process ID (PID)
- Process state
- Program counter
- CPU registers
- Memory management info
- I/O status
- Accounting information

Process Creation:

- **fork()**: Creates child process (copy of parent)
- **exec()**: Replaces process image with new program
- **wait()**: Parent waits for child completion

Inter-Process Communication (IPC):

1. **Shared Memory**: Processes share memory region
2. **Message Passing**: Processes communicate via messages
 - Direct/Indirect communication
 - Synchronous/Asynchronous communication
 - Buffering (zero, bounded, unbounded)

Threads:

- Lightweight processes sharing address space
- **User-level threads**: Managed by user library
- **Kernel-level threads**: Managed by OS
- **Benefits**: Responsiveness, resource sharing, economy, scalability

3. CPU Scheduling

Scheduling Objectives:

- Maximize CPU utilization and throughput
- Minimize turnaround time, waiting time, response time
- Ensure fairness

Key Metrics:

- **Arrival Time (AT)**: When process arrives
- **Burst Time (BT)**: CPU time required
- **Completion Time (CT)**: When process finishes
- **Turnaround Time (TAT)**: $CT - AT$
- **Waiting Time (WT)**: $TAT - BT$
- **Response Time (RT)**: First CPU allocation - AT

Scheduling Algorithms:

1. First Come First Serve (FCFS):

- Non-preemptive
- Simple but can cause convoy effect
- Average WT can be high

2. Shortest Job First (SJF):

- **Non-preemptive**: Select shortest job from ready queue
- **Preemptive (SRTF)**: Switch if shorter job arrives
- Optimal for average waiting time
- Suffers from starvation

3. Priority Scheduling:

- Higher priority processes executed first
- Can be preemptive or non-preemptive
- **Problem**: Starvation of low priority processes
- **Solution**: Aging (gradually increase priority)

4. Round Robin (RR):

- Time quantum (q) assigned to each process
- Preemptive
- Good response time
- Performance depends on time quantum size

5. Multilevel Queue:

- Separate queues for different process types
- Each queue has its own scheduling algorithm
- Fixed priority between queues

6. Multilevel Feedback Queue:

- Processes can move between queues
- Aging prevents starvation
- Most complex but flexible

GATE Tips for Scheduling:

- Always draw Gantt chart for numerical problems
- Calculate metrics systematically
- Remember SRTF gives minimum average WT

- For RR, if all processes have same AT, average WT = $(n-1) \times q / 2$

4. Process Synchronization

Critical Section Problem:

Section of code that accesses shared resources. Requirements:

1. **Mutual Exclusion**: Only one process in critical section
2. **Progress**: Decision on next entry can't be postponed indefinitely
3. **Bounded Waiting**: Limit on waiting time

Solutions:

1. Peterson's Algorithm:

For 2 processes using turn and flag variables.

2. Semaphores:

- **Binary Semaphore**: Values 0 or 1
- **Counting Semaphore**: Non-negative integer values
- Operations: wait(P) and signal(V)

3. Monitors:

High-level synchronization construct with condition variables.

4. Message Passing:

Using send() and receive() operations.

Classical Synchronization Problems:

Producer-Consumer Problem:

- **Buffer**: Shared between producer and consumer
- **Semaphores needed**: mutex, empty, full
- Producer waits when buffer full, consumer waits when empty

Reader-Writer Problem:

- Multiple readers can access simultaneously
- Writer needs exclusive access
- **Semaphores**: readcount, mutex, wrt

Dining Philosopher Problem:

- 5 philosophers, 5 chopsticks
- Demonstrates deadlock and starvation issues
- Solutions: Asymmetric approach, waiter solution

Important Concepts:

- **Race Condition**: Outcome depends on execution timing
- **Atomic Operations**: Indivisible operations

- **Busy Waiting**: Continuous checking (wasteful)
- **Spinlock**: Lock using busy waiting

5. Deadlock

Definition:

Set of processes blocked, each holding resources and waiting for resources held by others.

Necessary Conditions (Coffman Conditions):

1. **Mutual Exclusion**: Resources cannot be shared
2. **Hold and Wait**: Process holds resources while waiting for others
3. **No Preemption**: Resources cannot be forcibly taken
4. **Circular Wait**: Circular chain of waiting processes

Deadlock Handling Strategies:

1. Prevention:

Negate one of the four conditions:

- Mutual Exclusion: Make resources shareable (not always possible)
- Hold and Wait: Acquire all resources at once
- No Preemption: Allow resource preemption
- Circular Wait: Order resources, request in ascending order

2. Avoidance:

Use algorithms to avoid unsafe states:

- **Banker's Algorithm**: Check if allocation leads to safe state
- **Safe State**: Sequence exists where all processes can complete
- **Unsafe State**: May lead to deadlock

3. Detection and Recovery:

- Periodically check for deadlock using wait-for graph
- **Recovery methods**: Process termination, resource preemption

4. Ignorance:

Assume deadlock won't occur (used by most OS)

Banker's Algorithm Steps:

1. Check if request $\leq$ need
2. Check if request $\leq$ available
3. Tentatively allocate resources
4. Run safety algorithm
5. If safe, commit allocation; else rollback

6. Memory Management

Memory Hierarchy:

Registers → Cache → Main Memory → Secondary Storage

Address Binding:

- **Compile time**: Absolute addresses generated
- **Load time**: Relocatable addresses resolved at load
- **Runtime**: Address translation during execution

Memory Allocation Strategies:

Contiguous Allocation:

- **Fixed Partitioning**: Memory divided into fixed partitions
- **Variable Partitioning**: Partitions created dynamically

Allocation Algorithms:

1. **First Fit**: Allocate first suitable hole
2. **Best Fit**: Allocate smallest suitable hole
3. **Worst Fit**: Allocate largest suitable hole

Problems:

- **External Fragmentation**: Free memory scattered
- **Internal Fragmentation**: Allocated memory larger than needed

Paging:

- **Logical memory**: Divided into pages
- **Physical memory**: Divided into frames
- **Page Table**: Maps logical to physical addresses
- **Address Translation**: Page number + offset

Advantages:

- No external fragmentation
- Allows non-contiguous allocation

Page Table Implementation:

- Registers (small page tables)
- Memory (with PTBR - Page Table Base Register)
- TLB (Translation Lookaside Buffer) for fast access

Segmentation:

- Memory divided into variable-size segments
- Each segment has base and limit
- **Segment Table**: Contains base address and limit

Segmentation vs Paging:

- Segmentation: Variable size, logical division
- Paging: Fixed size, physical division

7. Virtual Memory

Concept:

Technique allowing execution of programs larger than physical memory by keeping only needed parts in memory.

Demand Paging:

- Pages loaded only when referenced
- ****Page Fault****: Reference to page not in memory
- ****Valid/Invalid bit****: Indicates if page is in memory

Page Replacement Algorithms:

1. FIFO (First In First Out):

- Replace oldest page
- Can suffer from Belady's anomaly

2. Optimal:

- Replace page that will be used farthest in future
- Theoretical optimum, not practical

3. LRU (Least Recently Used):

- Replace page used least recently
- Good performance but expensive to implement

4. LFU (Least Frequently Used):

- Replace page with lowest reference count

5. Second Chance (Clock):

- FIFO with reference bit
- Give second chance to referenced pages

Thrashing:

- Excessive paging activity
- ****Cause****: Too many processes, insufficient memory
- ****Solution****: Reduce multiprogramming degree

Working Set:

- Set of pages referenced by process in time window
- Used to determine memory requirements

Allocation Strategies:

- ****Equal allocation****: Equal frames to all processes
- ****Proportional allocation****: Frames based on process size
- ****Priority allocation****: Frames based on process priority

8. File Systems

File Concept:

Named collection of related information stored on secondary storage.

File Attributes:

- Name, type, size, location
- Protection, timestamps
- User identification

File Operations:

- Create, delete, open, close
- Read, write, seek, truncate

Directory Structure:

- **Single-level**: All files in one directory
- **Two-level**: Separate directory for each user
- **Tree**: Hierarchical structure
- **Acyclic graph**: Sharing allowed
- **General graph**: Links can create cycles

File Allocation Methods:

1. Contiguous Allocation:

- Files occupy consecutive blocks
- **Advantages**: Simple, good performance for sequential access
- **Disadvantages**: External fragmentation, file size changes difficult

2. Linked Allocation:

- File blocks linked using pointers
- **Advantages**: No fragmentation, dynamic size
- **Disadvantages**: Sequential access only, pointer overhead

3. Indexed Allocation:

- Index block contains pointers to data blocks
- **Advantages**: Direct access, no fragmentation
- **Disadvantages**: Index block overhead

Free Space Management:

- **Bit vector**: Bit for each block (0=free, 1=allocated)
- **Linked list**: Free blocks linked together
- **Grouping**: Store addresses of free blocks
- **Counting**: Store address and count of consecutive free blocks

Disk Scheduling:

Algorithms to minimize seek time:

1. **FCFS**: Service requests in arrival order
2. **SSTF**: Shortest Seek Time First
3. **SCAN**: Elevator algorithm, sweep across disk
4. **C-SCAN**: Circular SCAN, return to start after end
5. **LOOK/C-LOOK**: SCAN/C-SCAN without going to disk ends

9. I/O Systems

I/O Hardware:

- **Port**: Connection point for device
- **Bus**: Set of wires for communication
- **Controller**: Electronics that control device

I/O Methods:

1. Programmed I/O:

- CPU continuously polls device status
- Simple but CPU intensive

2. Interrupt-driven I/O:

- Device interrupts CPU when ready
- Better CPU utilization

3. DMA (Direct Memory Access):

- Device transfers data directly to/from memory
- CPU only involved at beginning and end

I/O Software Layers:

1. **Interrupt handlers**: Handle hardware interrupts
2. **Device drivers**: Device-specific software
3. **Device-independent I/O**: Common interface
4. **User-level I/O**: System calls and libraries

Buffering:

- **No buffering**: Direct transfer
- **Single buffering**: One buffer between OS and user
- **Double buffering**: Two buffers for overlap
- **Circular buffering**: Multiple buffers in circular fashion

10. Important Formulas and Quick Reference

CPU Scheduling Metrics:

- **Turnaround Time** = Completion Time - Arrival Time

- **Waiting Time** = Turnaround Time - Burst Time
- **Response Time** = First CPU Time - Arrival Time
- **CPU Utilization** = (Total Burst Time / Total Time) × 100%
- **Throughput** = Number of processes completed / Total Time

Memory Management:

- **Logical Address Space** = 2^m bytes (m-bit addresses)
- **Physical Address Space** = 2^n bytes (n-bit addresses)
- **Page Size** = 2^p bytes
- **Number of Pages** = Logical Address Space / Page Size
- **Number of Frames** = Physical Address Space / Frame Size

Virtual Memory:

- **Effective Access Time** = Hit Rate × Memory Access Time + Miss Rate × Page Fault Time
- **Page Fault Rate** = Number of Page Faults / Number of Memory References

Disk Performance:

- **Seek Time**: Time to move head to correct track
- **Rotational Latency**: Time to rotate to correct sector
- **Transfer Time**: Time to transfer data
- **Total Access Time** = Seek Time + Rotational Latency + Transfer Time

Key Points for GATE:

1. **Process States**: Remember the 5-state model
2. **Scheduling**: SRTF gives minimum average waiting time
3. **Synchronization**: Know semaphore operations and classical problems
4. **Deadlock**: Banker's algorithm implementation
5. **Paging**: Address translation formula = (page_number × page_size) + offset
6. **Page Replacement**: Belady's optimal algorithm is theoretical minimum
7. **File Systems**: Know allocation methods trade-offs
8. **Disk Scheduling**: SCAN family algorithms are most practical

Common GATE Question Types:

- Numerical problems on scheduling algorithms
- Address translation in paging/segmentation
- Deadlock detection and prevention scenarios
- Page replacement algorithm comparisons
- File allocation and disk scheduling calculations
- Process synchronization with semaphores

Study Strategy:

1. Start with basic concepts and definitions
2. Practice numerical problems extensively
3. Draw diagrams for complex concepts
4. Solve previous GATE questions
5. Focus on algorithm implementations
6. Understand trade-offs between different approaches